A Uniformly Distributed CORDIC-Based RoPE Hardware Accelerator for LLMs

***Abstract*—Rotary Positional Encoding (RoPE) is a fundamental mechanism in transformer-based large language models (LLMs) that captures positional information by applying trigonometric rotations. However, a straightforward hardware implementation of sine and cosine functions needed by RoPE requires considerable area and power resources, which becomes a serious bottleneck for edge inference performance. In this work, we propose a hardware-efficient RoPE accelerator based on Uniformly Distributed (UD) CORDIC architectures, completely eliminating the need for explicit trigonometric computations. Both Binary UD-CORDIC and Canonical Signed Digit (CSD) UD-CORDIC variants are explored and mapped to fully pipelined fixed-point datapaths. The proposed designs are implemented in parameterized RTL and synthesized using a 45 nm ASIC flow to evaluate area, power, and timing. Furthermore, rigorous end-to-end validation is performed by integrating the fixed-point hardware models into the native attention mechanisms of state-of-the-art LLMs, including LLaMA-2 (7B/13B), Mistral-7B, and Qwen2-7B. Experimental results demonstrate that across a fixed-point parameter sweep (6 to 14 fractional bits), the proposed Binary UD architecture reduces cell area by up to 12.6% and power by 33–37% compared to the same-stage standard CORDIC baseline. More impressively, the matched $N/2$-stage CSD UD-CORDIC consistently achieves area reductions ranging from 27.1% to 31.4% and power savings ranging from 62.3% to 64.5% compared to the N-stage standard CORDIC baseline. The system-level evaluation confirms that this aggressive hardware reduction introduces minimal degradation to language generation perplexity, defining the optimal boundary of trade-offs for Edge LLM accelerators. All the hardware design files are made freely available for easy adoption and further usage to the researchers and designers community.**

***Index Terms*—Large Language Models, Hardware Accelerator, CORDIC Architectures, Rotary Position Embedding, Edge AI, Fixed-Point Arithmetic.**

I. Introduction

Transformer-based large language models (LLMs) have achieved unparalleled performance across natural language processing, code generation, and complex reasoning tasks [1]. As these models scale, deploying them on resource-constrained Edge devices requires extreme hardware optimization [1]–[3]. A critical computational bottleneck in the modern transformer attention mechanism is Rotary Positional Encoding (RoPE). Unlike absolute positional embeddings, which are simply added to token vectors, RoPE injects positional information into token embeddings through structured rotations within the feature space [4], [5].

Conventional implementations rely on floating-point computation or lookup-table (LUT) based approaches for trigonometric evaluation, leading to increased hardware complexity and memory overhead [6]. Both approaches incur severe memory bandwidth bottlenecks and substantial silicon overhead, making them unsuitable for low-power Edge deployment [5].

The Coordinate Rotation Digital Computer (CORDIC) algorithm [7]–[9] offers an alternative for realizing trigonometric operations by replacing complex multipliers with simple shift-and-add logic. Traditional standard CORDIC implementations employ micro-rotations with mathematically strict angles of the form $\tan^{-1}(2^{-i})$. While this avoids multipliers, standard CORDIC requires a dedicated dynamic control path (the Z-path) to iteratively accumulate the angle and determine the direction of each subsequent micro-rotation [10]–[12].

Recently, the uniformly distributed (UD) CORDIC [13] was proposed to address these limitations by adopting a uniform angle set $\alpha_i = 2^{-i}$. This enables the extraction of micro-rotation directions directly from the binary representation of the rotation angle, eliminating the Z-path entirely. While UD-CORDIC has demonstrated architectural improvements for generic signal-processing rotations, its application to RoPE in LLMs, along with its hardware–accuracy trade-offs, remains largely unexplored.

This work introduces multiplier-free, Z-path-less RoPE accelerator designs based on Binary and CSD UD-CORDIC architectures. To correlate hardware efficiency with AI accuracy, we conduct an end-to-end fixed-point perplexity analysis of modern LLMs, determining the maximum hardware reduction that can be achieved without degrading language generation.

In addition to conventional numerical error metrics, we perform end-to-end evaluation by integrating the proposed RoPE approximations into transformer models and measuring their impact using perplexity. Furthermore, we analyze the effect of fixed-point precision by sweeping the fractional bit width and examining its impact on both hardware cost and model accuracy, identifying an optimal precision range for efficient deployment. All the hardware design files are made freely available at [14] for easy adoption and further usage to the researchers and designers community.

II. Proposed Methodology

A. Rotary Positional Embedding (RoPE) Formulation

The self-attention mechanism used in regular transformer architectures is naturally permutation-invariant, that is, it has no notion of order. RoPE globally rotates the queries and keys in a multi-dimensional space to inject positional awareness into the model. RoPE encodes position multiplicatively, enabling the model to capture approximate relative distances between tokens, in contrast to traditional absolute positional embeddings, which are merely added to input vectors.

Given an input embedding vector $\mathbf{x} \in \mathbb{R}^d$ representing a token at absolute sequence position m, RoPE divides the d-dimensional vector into $d/2$ mutually exclusive 2D subspaces, i.e adjacent elements of the embedding vector are grouped into pairs of the form (x_{2i}, x_{2i+1}). This representation allows the rotation operation to be expressed as a complex multiplication, enabling an efficient hardware realization using CORDIC-based architectures. A unique rotation is then applied to each adjacent pair of feature dimensions (x_{2i}, x_{2i+1}) [3], [4]:

$$\begin{bmatrix} x'_{2i} \\ x'_{2i+1} \end{bmatrix} = \begin{bmatrix} \cos(m\Theta_i) & -\sin(m\Theta_i) \\ \sin(m\Theta_i) & \cos(m\Theta_i) \end{bmatrix} \begin{bmatrix} x_{2i} \\ x_{2i+1} \end{bmatrix} \quad (1)$$

where $\theta_i = m\Theta_i$ represents the rotation angle. Here, m dictates the token's position in the sequence, and Θ_i is a predefined frequency base for the i-th dimension pair, typically defined as $\Theta_i = b^{-2i/d}$ with base b [3], [4].

From a hardware implementation perspective, this formulation reduces to a complex domain rotation. By treating the paired elements as the real and imaginary components of a complex number, the transformation simplifies to Equation 2.

$$x'_{complex} = (x_{2i} + jx_{2i+1}) \cdot e^{jm\Theta_i} \quad (2)$$

The architectural advantage of this rotation is recognized during attention scoring. While taking the inner product between a rotated query vector $\mathbf{q}$ at position m and a rotated key vector $\mathbf{k}$ at position n, the trigonometric transformation algebraically reduces to a function of their relative distance $(m-n)$:

$$\langle \mathbf{q}_m, \mathbf{k}_n \rangle = \text{Re}\left(\sum_{i=0}^{d/2-1} (\mathbf{q}_i e^{jm\Theta_i})(\mathbf{k}_i e^{jn\Theta_i})^* \right) \propto f(m-n) \quad (3)$$

However, when computing Equations 1 and 2, we implicitly depend on high-throughput trigonometric evaluations and intricate multiplications. These operations severely bottleneck traditional arithmetic logic units (ALUs) on Edge chips, directly prompting the need for multiplierless hardware accelerators.

B. Standard CORDIC-Based RoPE

The conventional approach to hardware-efficient rotation uses the CORDIC algorithm [6], [15]–[20], where a rotation is decomposed into a sequence of micro-rotations:

$$\theta = \sum_{i=0}^{M-1} \delta_i \tan^{-1}(2^{-i}), \quad \delta_i \in \{-1, 1\} \quad (4)$$

Each micro-rotation is implemented using shift-and-add operations. However, standard CORDIC requires iterative angle updates and control logic to compute δ_i, thereby increasing latency and hardware overhead [21], [22].

C. Binary UD-CORDIC

In standard CORDIC, the hardware must sequentially compute the remaining angle error at each step to determine whether the next rotation should be positive or negative (forming a closed-loop Z-path). To eliminate this, the Binary UD-CORDIC replaces the conventional arc-tangent angle set with a linear sequence $\alpha_i = 2^{-i}$.

Because the angle base is a simple power-of-two, the rotation angle θ can be treated as a direct, feed-forward instruction set. The rotation direction δ_i is explicitly extracted from the i-th bit of the binary-represented angle. For example, a binary bit of '1' dictates a positive rotation ($\delta_i = 1$), while a '0' dictates a negative rotation ($\delta_i = -1$).

$$\theta \approx \sum_{i=0}^{M-1} \delta_i \cdot 2^{-i}, \quad \delta_i \in \{-1, 1\} \quad (5)$$

Unlike conventional methods that allow a zero-rotation ($\delta_i = 0$), the binary UD-CORDIC restricts δ_i to strictly $\{-1, 1\}$. This is important because it ensures that the hardware maintains a constant-magnitude scaling factor across all datapath instances. By removing the Z-path, the iterative hardware equations for the datapath simplify to a simple, open-loop shift-and-add sequence:

$$x_{i+1} = x_i - \delta_i(y_i \gg (i+1)) \quad (6)$$

$$y_{i+1} = y_i + \delta_i(x_i \gg (i+1)) \quad (7)$$

D. CSD UD-CORDIC

While Binary UD-CORDIC removes the Z-path, the physical datapath still requires N sequential adders for an N-bit angle. To further improve this, Canonical Signed Digit (CSD) encoding is applied to θ [13], [17], [23]. CSD represents numbers using the digits $\{-1, 0, 1\}$ with the strict mathematical property that no two non-zero digits are adjacent. Because adjacent active rotations never occur, two consecutive Binary UD stages (stage i and stage $i+1$) are merged together. Instead of performing two sequential shift-and-add operations, the CSD stage mathematically pre-computes the combined rotation effect:

$$\theta \approx \sum_{i=0}^{M/2-1} \delta'_i, \quad \delta'_i \in \{-2, -1, 0, 1, 2\} \quad (8)$$

In hardware, this structural fusion means two sequential adders are replaced by a single adder. This allows a CSD UD-CORDIC pipeline to effectively halve the number of hardware stages ($N/2$), thereby drastically reducing switching activity and area [24], while achieving angular resolution comparable to that of a Binary UD pipeline with N stages.

III. Hardware Architecture and Implementation

A. RTL Datapath and Pipeline Organization

The proposed RoPE accelerator is implemented as a parameterized, fully pipelined register-transfer level (RTL) module [7], [12], [25]. Three architectural variants are evaluated: the standard CORDIC, the Binary UD-CORDIC, and the CSD UD-CORDIC.

In the Binary UD design, the target rotation angle is directly encoded as a bit vector. These control bits are propagated alongside the datapath via a parallel register pipeline, ensuring

that the alignment between the data and control logic is cycle-accurate. Each micro-rotation stage executes a deterministic update utilizing arithmetic right shifts. Because the shift magnitudes are statically defined per stage, they are realized entirely through hardwired routing, incurring zero physical gate overhead and eliminating the need for iterative feedback control [10], [26].

Conversely, the CSD UD-CORDIC design leverages the non-adjacent property of Canonical Signed Digits to map the angle to a signed-digit representation ($\{-2, -1, 0, 1, 2\}$), similar to radix-4 Booth encoding. This enables the mathematical combination of the rotation effects of multiple Binary stages into a single hardware block. Pipeline registers are inserted to break combinational paths, yielding CSD UD-CORDIC's $N/2$ pipelined stages, compared to the N-stage Binary UD pipeline.

B. Quantization and Synthesis Methodology

To ensure ASIC implementation efficiency, the design avoids floating-point arithmetic entirely and relies strictly on a parameterized fixed-point numerical representation [12]. All datapath operands and intermediate values are mapped to a Q$(1, F)$ format, comprising 1 sign bit, 1 integer bit, and F fractional bits, yielding a total datapath word length of $W = F + 2$. By parameterizing F and the pipeline stages (N), the RTL generators enable systematic exploration of the precision-area trade-off.

Hardware evaluations were conducted using Cadence Genus Synthesis Software mapped to a TSMC 45 nm standard cell library [9], [11]. All designs were synthesized using standard cells operating at a nominal supply voltage of 1.2 V under a fixed 500 MHz clock constraint. Post-synthesis reports were used to extract metrics on total cell area, power, and timing.

IV. Experimental Results and Analysis

We evaluate the architectures across three domains: mathematical algorithmic accuracy, fixed-point hardware synthesis, and end-to-end LLM perplexity validation.

A. Algorithm-Level Accuracy and Stage Equivalence

To isolate and verify the underlying rotation algorithms independent of the RoPE architecture, we evaluate the ground-truth algorithms using fixed-point arithmetic prior to hardware implementation to establish their absolute mathematical convergence limits. Table I reports the resulting Mean Squared Error (MSE) and Angular Error.

TABLE I
Algorithm-Level Accuracy versus Number of Stages

Method	Stages	MSE	Angle Err (rad)	Angle Err (°)
Binary UD	5	4.36×10^{-4}	0.0677	3.88°
Binary UD	10	1.72×10^{-4}	0.0384	2.20°
CSD UD	5	3.38×10^{-4}	0.0591	3.38°
CSD UD	10	3.09×10^{-4}	0.0547	3.13°

For Binary UD, increasing the number of stages from 5 to 10 noticeably improves angular accuracy. However, for CSD UD, the angular error reduction from 5 to 10 stages is a marginal 0.25°. This mathematically confirms that the $N/2$-stage CSD UD successfully captures the precision of the much deeper N-stage Binary UD through its non-adjacent-stage merging.

TABLE II
ASIC Synthesis Results for varying stages (Q1.7 Format).

Method	Stages	Area (μm^2)	Power (mW)	Slack (ns)
Std CORDIC [5]	8	2773	2.464	1.381
Binary UD [13]	8	2425	1.56	1.594
CSD UD [13]	4	1903	0.928	1.355
CSD UD (Ext.)	8	3769	2.640	1.331
Std CORDIC	10	3292	3.062	1.381
Binary UD	10	3068	1.999	1.584
CSD UD	5	2330	1.117	1.347
CSD UD (Ext.)	10	4820	3.225	1.352
Std CORDIC	12	3813	3.663	1.360
Binary UD	12	3742	2.4441	1.584
CSD UD	6	2779	1.299	1.353
CSD UD (Ext.)	12	5970	3.858	1.313

Across the evaluated stage configurations ($N = 8$ to 12), the N-stage Binary UD reduces area by up to 12.6% and power by 33–37% compared to the N-stage standard CORDIC. More impressively, the matched $N/2$-stage CSD UD architecture leverages the mathematical stage-merging property, achieving area reductions of 27.1–31.4% and power savings of 62.3–64.5% over the equivalent N-stage standard CORDIC baseline. The observed savings in area and power are primarily attributed to the elimination of the Z-path and simplified control logic in UD-CORDIC architectures.

B. Impact of Fractional Precision on Hardware Cost

To identify the saturation limits of hardware optimization, synthesis was swept from $F = 14$ down to $F = 6$ (Table III). As expected, the physical silicon area decreases monotonically with reduced precision for both architectures.

TABLE III
Area and Power versus Fractional Bit-Width (F).

Frac Bits (F)	Binary UD 10 Stages		CSD UD 5 Stages	
	Area (μm^2)	Power (mW)	Area (μm^2)	Power (mW)
14	4947	3.18	4025	2.10
13	4646	2.98	3766	1.99
12	4346	2.80	3467	1.76
11	4299	2.87	3218	1.64
10	3961	2.72	2942	1.47
9	3666	2.56	2734	2.15
8	3319	2.18	2605	1.24
7	3068	2.00	2330	1.18
6	2728	1.79	2051	0.94

C. System-Level Validation and Trade-off Analysis

The hardware logic models were dynamically injected into the native RoPE computations of multiple open-source LLMs [2], [3], [27] to ensure that the hardware reductions do not degrade the AI's contextual reasoning capabilities. The WikiText-2 dataset was used to evaluate inference and determine the language modeling perplexity (Table IV).

TABLE IV
PERPLEXITY VERSUS FRACTIONAL BIT-WIDTH (F) FOR BINARY UD (10-STAGE) AND CSD UD (5-STAGE) ROPE EVALUATED ON WIKITEXT-2.

Model	Variant	F=14	F=13	F=12	F=11	F=10	F=9	F=8	F=7	F=6	F=5	F=4	F=3
LLaMA-2 7B	Binary	6.1066	6.1066	6.1066	6.1066	6.1067	6.1066	6.1069	6.1078	6.1091	6.1217	6.1648	6.3872
	CSD	6.1067	6.1066	6.1067	6.1066	6.1066	6.1067	6.1066	6.1072	6.1097	6.1169	6.1476	6.2964
LLaMA-2 13B	Binary	5.4497	5.4497	5.4497	5.4497	5.4497	5.4496	5.4498	5.4505	5.4529	5.4610	5.4977	5.6783
	CSD	5.4497	5.4497	5.4497	5.4497	5.4497	5.4497	5.4497	5.4501	5.4516	5.4579	5.4820	5.5930
Falcon-7B	Binary	7.2547	7.2547	7.2546	7.2546	7.2546	7.2547	7.2547	7.2548	7.2565	7.2606	7.2794	7.3605
	CSD	7.2546	7.2546	7.2546	7.2546	7.2546	7.2546	7.2547	7.2549	7.2551	7.2590	7.2711	7.3265
Gemma-2 9B	Binary	6.3881	6.3881	6.3882	6.3881	6.3882	6.3882	6.3881	6.3881	6.3893	6.3912	6.4013	6.4489
	CSD	6.3881	6.3882	6.3881	6.3882	6.3881	6.3881	6.3882	6.3884	6.3889	6.3911	6.3982	6.4306
Mistral-7B	Binary	5.8927	5.8927	5.8927	5.8927	5.8927	5.8927	5.8928	5.8929	5.8934	5.8972	5.9106	5.9653
	CSD	5.8927	5.8927	5.8927	5.8927	5.8927	5.8927	5.8927	5.8928	5.8934	5.8952	5.9067	5.9424
PHI-3 (3.8B)	Binary	5.7339	5.7339	5.7339	5.7342	5.7340	5.7338	5.7338	5.7344	5.7347	5.7545	5.7789	5.9269
	CSD	5.7339	5.7340	5.7339	5.7339	5.7340	5.7339	5.7341	5.7343	5.7358	5.7427	5.7651	5.8632
QWEN2-7B	Binary	7.8603	7.8602	7.8603	7.8601	7.8605	7.8603	7.8610	7.8617	8.2065	21.302	391.78	3461.5
	CSD	7.8604	7.8603	7.8602	7.8602	7.8603	7.8603	7.8601	7.8613	7.9560	13.651	180.05	2410.0

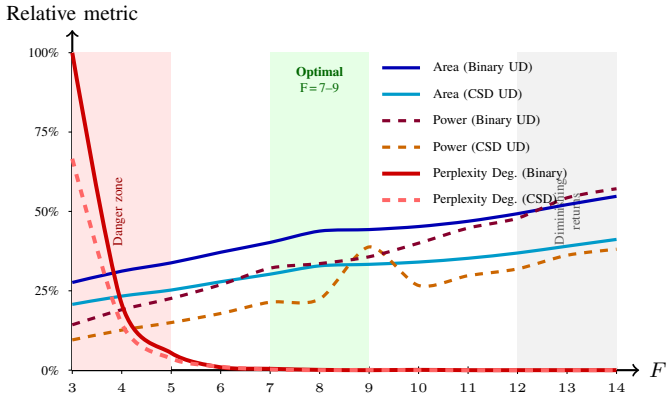

Fig. 1. Hardware–accuracy tradeoff across fractional bit-widths F (normalised to respective maxima). The silicon area and power scale monotonically with F for both the N-stage Binary and $N/2$-stage CSD UD variants, while perplexity degradation drops steeply as F increases and saturates near baseline at $F=7$. The shaded green band ($F=7$–9) indicates the optimal operating region: hardware cost is substantially reduced while perplexity remains indistinguishable from that of floating-point inference. The red zone ($F \leq 5$) indicates the onset of accuracy degradation, illustrated here using LLaMA-2 7B.

At fractional bits $F \leq 5$, there is a significant drop in LLM perplexity scores, which is prominent in Qwen2-7B. However, as fractional precision increases, perplexity rapidly stabilizes. At $F = 7$, the perplexity of quantizing models hits an absolute saturation point, becoming functionally identical to the baseline floating-point models across all parameter sizes (from 3.8B up to 13B). Intersecting this finding with the hardware metrics in Table III confirms that $F = 9$ (Total Word Length 11) is the optimal trade-off boundary. It significantly reduces hardware requirements without sacrificing LLM accuracy. Additionally, the generative accuracy of the N-stage Binary UD is consistently matched by the $N/2$-stage CSD UD. In contrast to prior RoPE accelerator designs [1], [5], this work provides a comprehensive evaluation of hardware–accuracy tradeoffs, validated at both the algorithmic level and across multiple LLMs.

V. CONCLUSION

This paper presents an area-efficient hardware architecture for LLM Rotary Position Embedding that uses Uniformly Distributed CORDIC algorithms. By deriving micro-rotation directions directly from the binary representation of the target angle, the proposed architectures completely eliminate the costly Z-path control logic found in standard CORDIC implementations [13]. Further, by leveraging Canonical Signed Digit (CSD) encoding, the CSD UD-CORDIC merges consecutive binary stages, allowing a highly truncated $N/2$-stage pipeline to match the mathematical accuracy of an N-stage Binary UD architecture. Exhaustive fixed-point LLM analysis across multiple modern architectures revealed that 9 fractional bits ($F = 9$) is the absolute optimal hardware configuration, preserving the LLM perplexity while minimizing physical silicon footprint. This establishes a highly robust architectural pathway for deploying energy-efficient Transformer inference on Edge AI processors. While the proposed architectures significantly reduce hardware cost, they rely on fixed-point quantization, which may introduce errors at extremely low precision levels. Future work will focus on extending the proposed framework to adaptive precision schemes for workload-aware optimization [21], [22], integrating the RoPE accelerator within a complete Transformer inference pipeline, exploring floating-point and mixed-precision implementations to mitigate quantization effects at very low precision [16], [18], [28], [29], and investigating algorithm–hardware co-design techniques to further reduce computational redundancy. All the hardware design files are made freely available at [14] for easy adoption and further usage to the researchers and designers community.

REFERENCES

[1] C.-T. Chen, H. Mun, J. Meng, M. S. Abdelfattah, and J.-s. Seo, "Hybrid systolic array accelerator with optimized dataflow for edge large language model inference," in *2025 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*, 2025, pp. 1–7.

[2] G. Islamoglu, M. Scherer, G. Paulin, T. Fischer, V. J. Jung, A. Garofalo, and L. Benini, "Ita: An energy-efficient attention and softmax accelerator for quantized transformers," in *2023 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*, 2023, pp. 1–6.

[3] Y. Jeon, M. Jang, H. Lee, Y. Jung, J. Jung, J. Lee, J. So, and D. Kim, "Ropim: A processing-in-memory architecture for accelerating rotary positional embedding in transformer models," *IEEE Computer Architecture Letters*, vol. 24, no. 1, pp. 41–44, 2025.

[4] Z. Huang and M. Chen, "Optimizing the learnable rope theta parameter in transformers," *IEEE Access*, vol. 13, pp. 131 271–131 288, 2025.

[5] W. Li, G. Wang, D. Lyu, N. Xu, and G. He, "Efficient hardware architecture design for rotary position embedding of large language models," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 15, no. 2, pp. 244–257, 2025.

[6] Z. Yin-Di, L. Ming, and W. Ming-Jiang, "A hardware modeling of high precision floating point arcsine/arccosine function based on cordic algorithm," in *2020 IEEE 5th International Conference on Signal and Image Processing (ICSIP)*, 2020, pp. 1040–1044.

[7] K.-D. Nguyen, D. T. Kiet, T.-T. Hoang, N. Q. N. Quynh, and C.-K. Pham, "A cordic-based trigonometric hardware accelerator with custom instruction in 32-bit risc-v system-on-chip," in *2021 IEEE Hot Chips 33 Symposium (HCS)*, 2021, pp. 1–13.

[8] P. A. Kumar, "Fpga implementation of the trigonometric functions using the cordic algorithm," in *2019 5th International Conference on Advanced Computing Communication Systems (ICACCS)*, 2019, pp. 894–900.

[9] W. Hong, H. Chen, L. Quan, Y. Fu, and L. Li, "Low-cost high-precision architecture for arbitrary floating-point nth root computation," in *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2023, pp. 1–5.

[10] H. Li and Y. Xin, "Modified cordic algorithm and its implementation based on fpga," in *2010 Third International Conference on Intelligent Networks and Intelligent Systems*, 2010, pp. 618–621.

[11] A. Changela, M. Zaveri, and A. Lakhlani, "Asic implementation of high performance radix-8 cordic algorithm," in *2018 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, 2018, pp. 699–705.

[12] Y. Moses and M. Rao, "A fixed-point pre-processing hardware architecture design for complex independent component analysis," in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2024, pp. 244–249.

[13] M. Garrido, D. Medina, P. Paz, and M. López-Vallejo, "Uniformly distributed cordic," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 72, no. 11, pp. 6948–6961, 2025.

[14] https://github.com/anon-researcher-2026-create/A-Uniformly-Distributed-CORDIC-Based-RoPE-Hardware-Accelerator-for-LLMs.

[15] P. Sharma, S. Nath, K. Guha, and K. L. Baishnab, "Design of low-latency, area- and power-efficient cordic algorithms for sine/cosine floating-point computation," in *2025 Devices for Integrated Circuit (DevIC)*, 2025, pp. 583–588.

[16] Y. Chang, P. Jokic, S. Emery, and L. Benini, "An ultra-low-power serial implementation for sigmoid and tanh using cordic algorithm," in *2023 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2023, pp. 1–2.

[17] Y. Zhang, L. Peng, L. Quan, Y. Zhang, S. Zheng, and H. Chen, "High-precision method and architecture for base-2 softmax function in dnn training," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 70, no. 8, pp. 3268–3279, 2023.

[18] H. Chen, L. Jiang, Y. Luo, Z. Lu, Y. Fu, L. Li, and Z. Yu, "A cordic-based architecture with adjustable precision and flexible scalability to implement sigmoid and tanh functions," in *2020 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2020, pp. 1–5.

[19] P. Paz and M. Garrido, "Cordic-based computation of arcsine and arccosine functions on fpga," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 70, no. 9, pp. 3684–3688, 2023.

[20] X. Liu, Y. Xie, H. Chen, and B. Li, "Implementation on fpga for cordic-based computation of arcsine and arccosine," in *IET International Radar Conference 2015*, 2015, pp. 1–4.

[21] O. Kokane, G. Raut, S. Ullah, M. Lokhande, A. Teman, A. Kumar, and S. K. Vishvakarma, "Retrospective: A cordic based configurable activation function for nn applications," in *2025 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, vol. 1, 2025, pp. 1–6.

[22] G. Raut, S. Rai, S. K. Vishvakarma, and A. Kumar, "A cordic based configurable activation function for ann applications," in *2020 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 2020, pp. 78–83.

[23] C. Paramasivam, S. S. Chauhan, V. Meena, A. Sreejagathi, B. A. V. N. Hasini, K. L. K. Kishore, T. V. N. G. Vamsikrishna, M. D. A. Sai, and A. Didi, "Enhanced performance of new scaling-free cordic for memory-based fast fourier transform architecture," *IEEE Access*, vol. 13, pp. 19 828–19 844, 2025.

[24] A. Verma, K. Kiyawat, B. P. Das, and P. K. Meher, "An efficient scaling-free folded hyperbolic cordic design using a novel low-complexity power-of-2 taylor series approximation," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 31, no. 8, pp. 1167–1177, 2023.

[25] Chetana and S. K P, "Vlsi implementation of coordinate rotation based design methodology using verilog hdl," in *2023 Third International Conference on Artificial Intelligence and Smart Energy (ICAIS)*, 2023, pp. 1574–1581.

[26] T. K. Rodrigues and E. E. Swartzlander, "Adaptive cordic: Using parallel angle recoding to accelerate rotations," *IEEE Transactions on Computers*, vol. 59, no. 4, pp. 522–531, 2010.

[27] S. Mehra, G. Raut, R. D. Purkayastha, S. K. Vishvakarma, and A. Biasizzo, "An empirical evaluation of enhanced performance softmax function in deep learning," *IEEE Access*, vol. 11, pp. 34 912–34 924, 2023.

[28] M. Basavaraju, V. Rayapati, and M. Rao, "Exploring hardware activation function design: Cordic architecture in diverse floating formats," in *2024 25th International Symposium on Quality Electronic Design (ISQED)*, 2024, pp. 1–8.

[29] V. Rayapati, S. G. Reddy, G. A. Kumar, G. R. Reddy, and M. Rao, "Ebaca: Efficient bfloat16-based activation function implementation using enhanced cordic architecture," in *2024 37th International Conference on VLSI Design and 2024 23rd International Conference on Embedded Systems (VLSID)*, 2024, pp. 605–610.